Exercise 07: *PyUVM* Universal Verification Components integration

Objective: *Introduction to reusable PyUVM Universal Verification Components.*

Task: *Updating the existing Saturation Filter filter testbench to drive and observe its sSDT interfaces with producer and consumer UVC agents.*

Start from your completed *Exercise 06: PyUVM Environment, Sequencer, and Sequences* testbench and copy it to `<ROOT>/lecture04/exercises/E07_sat_pyuvm_uvc`. In this exercise, you will replace the direct interface access with the provided *sSDT UVC*.

See `<ROOT>/lectures/common/docs/exercise_general_info.pdf` for more information on the *sSDT UVC*.

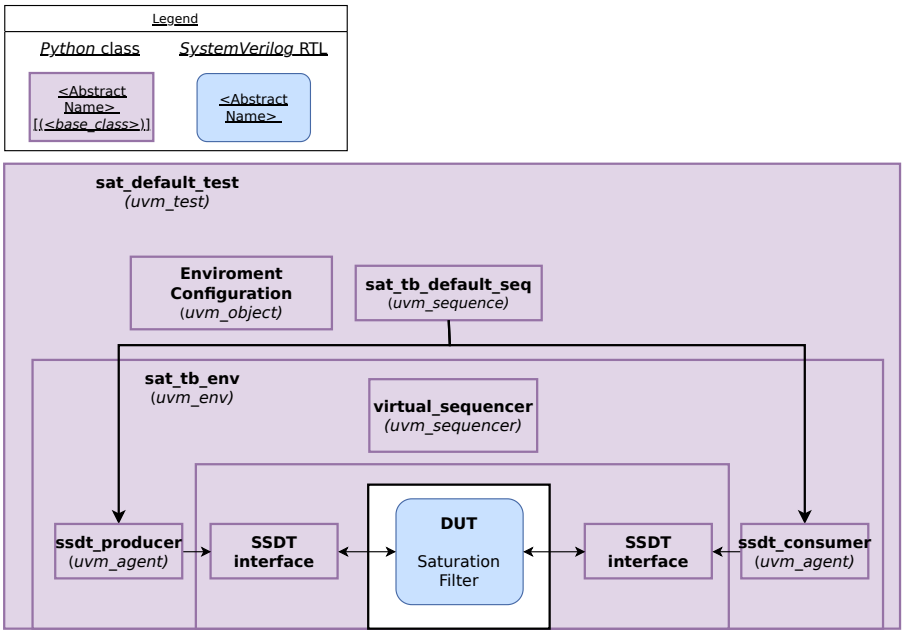


Fig. 1: Testbench overview with producer and consumer *UVC* agents.

The *Saturation Filter* testbench uses two agent instances:

- A *producer agent* drives the DUT input *sSDT* interface.
- A *consumer agent* waits for transactions on the DUT output *sSDT* interface.

Part 1: Testbench Configuration

First, we update the testbench configuration and replace the direct interface handles with configuration objects for the producer and consumer *UVC* agents.

In `sat_tb_config.py`,

- Import `uvc_ssdt_config` from `uvc.ssdt`.

```
from uvc.ssdt import uvc_ssdt_config
```

- In the class constructor replace the interface handles with two `uvc_ssdt_config` instances. Create the configurations using the factory.
  - `ssdt_prod_cfg` for the producer agent.
  - `ssdt_cons_cfg` for the consumer agent.
- After creating the configurations, configure the agents:
  - Set both `is_active` fields to `uvm_active_passive_enum.UVM_ACTIVE`.
  - Set `ssdt_prod_cfg.driver_type` to `uvc_ssdt_type_enum.PRODUCER`.
  - Set `ssdt_cons_cfg.driver_type` to `uvc_ssdt_type_enum.CONSUMER`.

In `sat_tb_default_test.py`

- Import `ssdt_interface_wrapper` from `uvc.ssdt` and replace the old interface wrapper type.
- In the `build phase`:
  - Assign the input and output interface handles to `cfg.ssdt_prod_cfg.vif` and `cfg.ssdt_cons_cfg.vif`, respectively.
  - Keep the `ConfigDB set` call for the environment and update the `connect_phase` variable names as needed.

Part 2: Producer and Consumer Agents

Next, we instantiate the producer and consumer agents in the environment and provide each agent with its corresponding configuration object.

In `sat_tb_env.py`:

- In the class constructor:
  - Create an empty handle `uvc_ssdt_producer` for the producer agent.
  - Create an empty handle `uvc_ssdt_consumer` for the consumer agent.
- In `build_phase` after retrieving the configuration and creating the virtual sequencer:
  - Create `uvc_ssdt_producer` using the factory.
  - Store `cfg.ssdt_prod_cfg` in the `ConfigDB` for the producer agent using the field name `cfg`.

```
ConfigDB().set(
    context     = self,
    inst_name   = "uvc_ssdt_producer",
    field_name  = "cfg",
    value       = self.cfg.ssdt_prod_cfg,
)
```

  - Create `uvc_ssdt_consumer` using the factory.
  - Store `cfg.ssdt_cons_cfg` in the `ConfigDB` for the consumer agent using the field name `cfg`.

Part 3: Virtual Sequencer Connections

Next, we connect the producer and consumer agent sequencers to the virtual sequencer so that virtual sequences can control both agents.

In `sat_tb_virt_sequencer.py`:

- In the class constructor, add two empty handles:
  - `ssdt_producer_sequencer` for the producer agent's sequencer.
  - `ssdt_consumer_sequencer` for the consumer agent's sequencer.
- Keep the configuration handle and its `ConfigDB` retrieval.

In `sat_tb_env.py`:

- Add a `connect_phase`.
- Connect each agent sequencer to its virtual sequencer handle:
  - Assign `uvc_ssdt_producer.sequencer` to `virtual_sequencer.ssdt_producer_sequencer`.

```
self.virtual_sequencer.ssdt_producer_sequencer = self.uvc_ssdt_producer.sequencer
```

  - Assign `uvc_ssdt_consumer.sequencer` to `virtual_sequencer.ssdt_consumer_sequencer`.

Part 4: Transaction-Level Virtual Sequence

Lastly, we replace the direct interface access in the virtual sequence with transaction-level producer and consumer *UVC* sequences.

In `sat_tb_default_seq.py`:

- Replace direct interface access with sequences on the *sSDT UVC* sequencers.
- Import `uvc_ssdt_basic_seq` and `uvc_ssdt_seq_item` from `uvc.ssdt`.
- In the sequence constructor, create these sequences using the factory:
  - A producer `uvc_ssdt_basic_seq`.
  - A consumer `uvc_ssdt_basic_seq`.

- For each transaction in `body`:
  - Create a `uvc_ssdt_seq_item` using the factory.
  - Assign the input value to its `data` field.
  - Assign the item to the producer sequence's `seq_item` field.

```
seq_item = uvc_ssdt_seq_item.create("producer_item")
seq_item.data = 46
self.producer_seq.seq_item = seq_item
```

  - Start the producer sequence on `self.sequencer.ssdt_producer_sequencer` and the consumer sequence on `self.sequencer.ssdt_consumer_sequencer`.
  - Run both sequences concurrently with `cocotb.start_soon()` and wait for both with `Combine`.

```
prod_task = cocotb.start_soon(
    self.producer_seq.start(
        seqr=self.sequencer.ssdt_producer_sequencer
    )
)
cons_task = cocotb.start_soon(
    self.consumer_seq.start(
        self.sequencer.ssdt_consumer_sequencer
    )
)

await Combine(prod_task, cons_task)
```

- Send one saturating and one non-saturating transaction. Reuse the sequences by assigning a new sequence item for the second transaction.

Part 5: Running the Test

Finally, we run the updated testbench and verify that both transaction types complete successfully through the *UVC* agents.

- Run the test using the `Makefile`.
- Verify that the non-saturating and saturating transactions pass.